Prathamesh Brahma

CS23179

Lab 7 Extra Datasets

Aim:To apply Support Vector Machine for Regression on Students Performance Dataset and analyze the impact of different kernels on model accuracy.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
path = "/content/drive/MyDrive/ML datasets/Student_Performance_dataset_5000.csv"
df = pd.read_csv(path)
df.head()
```

	student_id	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep_h
0	1	18	Other	High School	7.64	1.56	2.20	3.05	2.19	
1	2	18	Other	High School	2.21	2.22	2.10	1.65	2.55	
2	3	22	Male	High School	3.45	0.00	0.29	1.34	2.08	
3	4	17	Other	High School	5.75	2.08	3.01	2.27	2.20	
4	5	19	Other	High School	6.83	1.72	3.33	2.65	0.70	

5 rows × 21 columns

```
print(df.shape)
print(df.columns)
print(df.isnull().sum())
```

```
(5000, 21)
Index(['student_id', 'age', 'gender', 'academic_level', 'study_hours',
       'self_study_hours', 'online_classes_hours', 'social_media_hours',
       'gaming_hours', 'sleep_hours', 'screen_time_hours', 'exercise_minutes',
       'caffeine_intake_mg', 'part_time_job', 'upcoming_deadline',
       'internet_quality', 'mental_health_score', 'focus_index',
       'burnout_level', 'productivity_score', 'exam_score'],
      dtype='object')
student_id            0
age                   0
gender                0
academic_level        0
study_hours           0
self_study_hours      0
online_classes_hours  0
social_media_hours    0
gaming_hours          0
sleep_hours           0
screen_time_hours     0
exercise_minutes      0
caffeine_intake_mg    0
part_time_job         0
upcoming_deadline     0
internet_quality      0
mental_health_score   0
focus_index           0
burnout_level         0
productivity_score    0
exam_score            0
dtype: int64
```

```
df["Result"] = df["exam_score"].apply(lambda x: 1 if x >= 40 else 0)
```

```
X = df.drop(columns=['exam_score', 'Result'])
y = df['Result']
```

```
X = pd.get_dummies(X, drop_first=True)
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

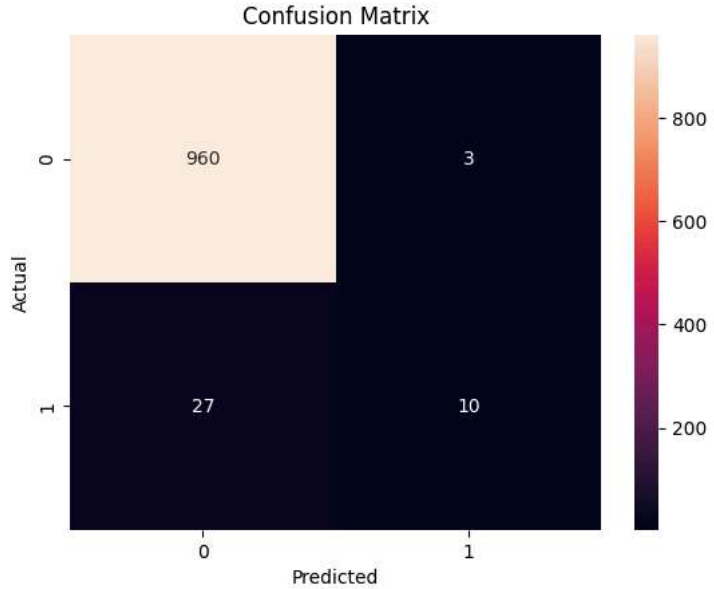
```
from sklearn.linear_model import LogisticRegression

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

print("Logistic Accuracy:", accuracy_score(y_test, y_pred_log))
```

```
Logistic Accuracy: 0.973
```

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.97
```

```
cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



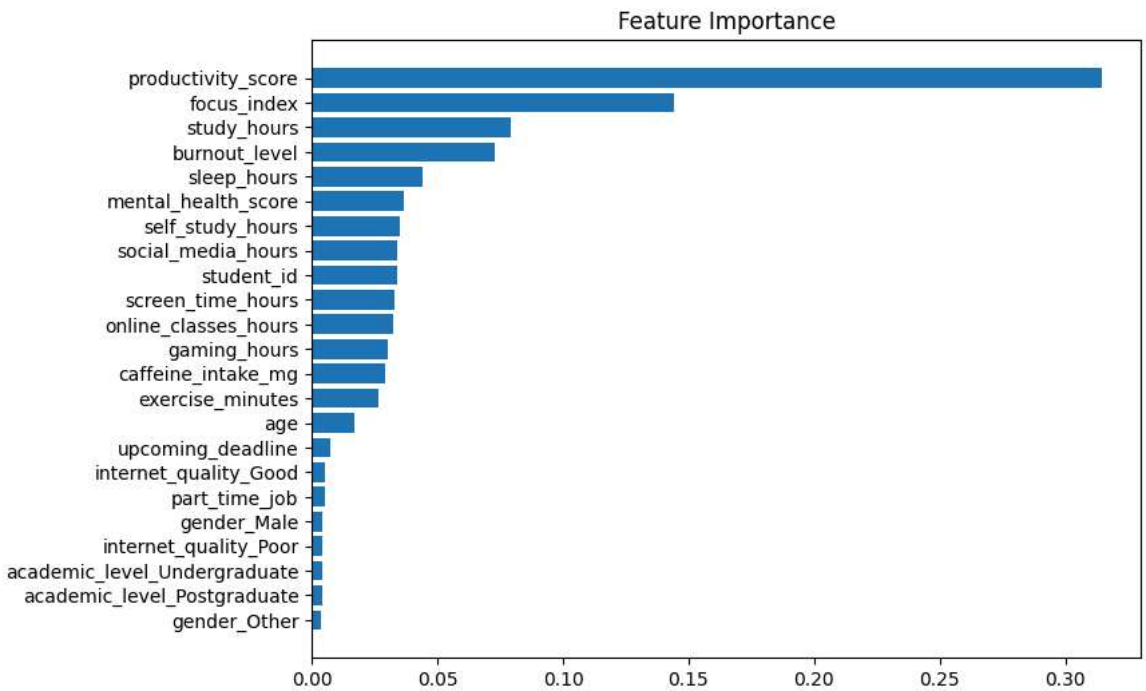

```
print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.98       963
           1       0.77      0.27      0.40        37

    accuracy                           0.97      1000
   macro avg       0.87      0.63      0.69      1000
weighted avg       0.97      0.97      0.96      1000
```

```
scores = cross_val_score(rf_model, X_scaled, y, cv=5)
print("Cross Validation Accuracy:", scores.mean())
```

```
Cross Validation Accuracy: 0.9648
```

```
importances = rf_model.feature_importances_
features = X.columns
indices = np.argsort(importances)
plt.figure(figsize=(8,6))
plt.barh(range(len(indices)), importances[indices])
plt.yticks(range(len(indices)), features[indices])
plt.title("Feature Importance")
plt.show()
```




```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
from sklearn.svm import SVC
svc_model = SVC(kernel='rbf')
svc_model.fit(X_train, y_train)
```

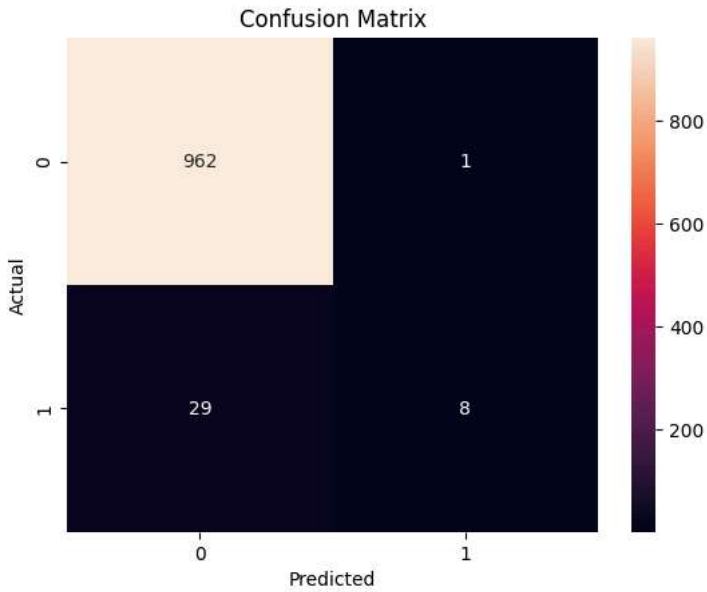
```
▾ SVC
SVC()
```

```
y_pred = svc_model.predict(X_test)
```

```
print("SVC Accuracy:", accuracy_score(y_test, y_pred))
```

```
SVC Accuracy: 0.97
```

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.97      1.00      0.98       963
           1       0.89      0.22      0.35        37

    accuracy                           0.97      1000
   macro avg       0.93      0.61      0.67      1000
weighted avg       0.97      0.97      0.96      1000
```